Domain 4: Prompt Engineering & Structured Output — Self-Questioning List

CCAR-F Prep — Domain 4 (20% of scored content).

A. Foundations of Prompt Engineering for Production

1. What is the difference between what satisfies a casual user (one good answer) and what a production system requires (the same behavior every time, across thousands of runs)?
2. Why is prompt engineering described as "the fastest and cheapest lever available" compared to other interventions?
3. What is the difference between a probabilistic control and a deterministic mechanism, and why does a "good prompt" only ever move the odds rather than guarantee an outcome?

What is Prompt Engineering?

4. How does prompt engineering differ from traditional software, where code executes exactly as written?
5. Why is the goal described as "directing the model's reasoning," not replacing it?

How It Works

6. Why does prompt engineering take effect immediately, with no retraining or deployment pipeline?
7. What is the five-step iterative process for developing a production prompt (write, test, identify failures, refine, repeat)?
8. Why must a prompt be tested "across varied examples" rather than just the easy cases — what is the actual goal (works once vs. holds across the full range of inputs)?

Zero-Shot and Few-Shot Prompting

9. What determines whether zero-shot is sufficient — is it about the task's difficulty, or about whether the correct output is easy to describe in words?
10. In the "Flag the important issues" example, why does the model's decision about what "important" means change from run to run?
11. Why does the advantage of few-shot only show up when the task is ambiguous or the output format is highly specific?

How Claude Interprets Instructions

12. Does Claude read only the sentence relevant to a specific concern, or does it form one overall sense of intent from the entire prompt?
13. Why can a stray phrase elsewhere in the prompt "push behavior in a direction you did not plan"?
14. Why do contradictory instructions "pull the model two ways and lower reliability" rather than simply being ignored?
15. How can a single keyword in a system prompt bias which tool the model chooses?

The Context Window

16. What happens to content that falls outside the context window — does it still influence the response?
17. Why can attention "spread thin" in very long prompts, causing key instructions to lose influence even though they're technically present?
18. Why is repeating a critical instruction near the start and again near the end of a long prompt considered a valid strategy despite the added token cost?

Intent versus Interpretation

19. What happens when a prompt leaves something unsaid — does the model ask for clarification, or construct a plausible guess?
20. Why does nothing in the model's response signal that an assumption was made when it fills a gap?
21. For "Flag the important issues," "Review this code," "Write an explanation," and "Keep it concise" — what specific kind of ambiguity does each example illustrate (undefined criteria, ambiguous scope, unstated audience, vague length)?

Prompt-Level Fix vs. Architectural Fix

22. What is the difference in how a prompt-level fix works (reducing ambiguity) versus an architectural fix (removing the model's discretion entirely)?
23. What is the single deciding question for choosing between them — "can this ever be allowed to fail?"
24. What happens when a prompt is used where a guarantee is actually required — does the gap disappear, or does it eventually fail?
25. What happens when architecture (a code gate, schema validator, required pipeline step) is added where a clearer prompt would have worked?

The Limits of Prompting

26. Why can no amount of "always do this" phrasing turn a probability into a guarantee?
27. In the vague-vs-explicit code review example, what specific elements does the explicit version add (what to report, what to skip, exact output shape) that the vague version lacks?

B. Designing Prompts with Explicit Criteria

28. What is the core difference between explicit criteria (concrete rules) and general guidance (a modifier on the model's own judgment)?
29. Why is the problem with vague instructions not that the model ignores them, but that it follows them using unstable judgment?

Why "Be Conservative" Does Not Work

30. Why does asking the model to "be conservative" or "only report high-confidence findings" usually fail to reduce noise?
31. Why is the model often already confident in the very cases it gets wrong — what does asking for more confidence fail to filter out?

32. Why is filtering by a confidence score unreliable specifically on hard cases?
33. What does naming explicit categories of what to report versus ignore give the model that "be conservative" does not?

Severity Classification

34. What is a severity level, and why does it replace a vague label with a rule?
35. Why can the same issue land in "major" on one run and "minor" on the next without anchoring examples for each tier?

False Positives and User Trust

36. Why does a category that frequently fires on non-issues "poison trust in the whole tool" rather than just that one category?
37. What is the recovery process when a category is found to be noisy (temporarily suppress it, improve its criteria, then reactivate)?

C. Few-Shot Prompting

38. What is the difference between a written rule describing what you want and an example showing it?
39. What three things does the model pick up simultaneously from two or three finished examples (format, level of detail, judgment calls)?

Few-Shot vs. Instruction-Only Prompting

40. What is the deciding question for whether zero-shot is sufficient — is the correct output easy to describe in words?
41. Why is few-shot prompting a common (incorrect) fix reached for when the actual problem is tool misrouting from weak tool descriptions?

Why Examples Outperform Longer Instructions

42. What is the functional difference between the model having to "imagine what a correct answer looks like" (zero-shot) versus "copy the demonstrated pattern" (few-shot)?
43. How do examples reduce made-up or empty fields in extraction tasks specifically?
44. What kind of implicit information (tone, level of detail, handling missing data) do examples carry that would take many words to state explicitly?

Anatomy of an Effective Example

45. What three qualities does Anthropic's guidance say good examples need (relevant, diverse, clear)?
46. Why do examples need to be diverse enough that the model doesn't "lock onto a pattern you did not intend"?
47. What is the correct way to structure multiple examples using `<example>` and `<examples>` tags?

Key Principles for Example Design

48. Why should examples show the reasoning behind a choice, not just the final answer?

49. Why are two to four examples resolving genuinely tricky cases worth more than many examples of the obvious case?
50. In extraction work, why should examples show "the same fact appearing in different document layouts"?

How Many Examples Is Enough

51. What happens with zero examples (the model invents format and judgment with no reference point)?
52. Why does a single example risk "creating an unintended pattern" since there's only one case to generalize from?
53. Why are 3 to 5 examples described as Anthropic's documented sweet spot?
54. Why do returns diminish beyond 5 examples on most tasks?

D. Structured Output with Tool Use and JSON Schemas

55. Why can't a downstream system reliably parse free-form text the way it can parse a guaranteed structured field?

Tool Use

56. What is tool use's role as "the primary mechanism for getting structured output"?
57. What are the three parts of a tool definition relevant to structured output (name, description, input schema)?

How the Tool Use Loop Works

58. What does Claude respond with instead of plain text when a tool is available?
59. What is the sequence of `stop_reason: tool_use` → tool execution → `tool_result` → continuation or `stop_reason: end_turn`?
60. For extraction tasks, how does the tool's input schema function as "the output contract"?

Why Tool Descriptions Matter

61. Does the model decide whether to call a tool based only on the instruction in the message, or primarily on how the tool itself is described?
62. Why should tool descriptions be written "with the same care as prompt instructions"?

JSON Schema

63. What does a JSON schema declare that a tool alone does not (fields, types, required-ness)?
64. What is the difference between "structured output" (forced shape) and "strict tool use" (`strict: true`, guaranteed match)?

JSON Schema Field Types

65. What is the behavioral difference between `required`, `optional`, `nullable`, and `enum` field types?

66. Why does marking a field required when the source might not contain it force the model to invent a value?
67. Why is nullable described as "the safer default for most optional data" compared to plain optional?
68. Why does an enum field prevent the model from inventing category labels or returning slight variations across runs?

The tool_choice Setting

69. What is the behavioral difference between the four `tool_choice` modes (`auto`, `any`, `tool`, `none`)?
70. When is `any` the right choice — what situation involves multiple possible schemas but an unknown input type?
71. When is `tool` the right choice — what kind of step "must always run first"?

Syntax Errors vs. Semantic Errors

72. What is the fundamental distinction between a syntax error (structural) and a semantic error (meaning)?
73. Why does a schema guarantee the shape of output but say nothing about whether the values inside make sense?
74. For each error type (missing closing brace, line items not summing to total, value in wrong field, contradicting fields), which are caught by schema validation and which are not?
75. Why is assuming that "strict tool use" or schema enforcement also guarantees correct *values* described as one of the most common production mistakes?

Native Structured Outputs Feature

76. What is the difference between the two modes of Structured Outputs — JSON outputs (`output_config.format`) and strict tool use (`strict: true`)?
77. When is JSON outputs the right mode (the response itself is the deliverable) versus strict tool use (each action in an agentic flow needs schema-valid inputs)?
78. Why does strict tool use matter more in a multi-step pipeline — what happens if wrong data flows into an early tool call?

E. Schema Design for Reliable Extraction

79. Why does the same model, given a slightly different schema, either invent a value or honestly report something is missing?
80. Why is schema design "not just about structure" but "about giving the model a way to be accurate"?

Required vs. Optional vs. Nullable Fields

81. Why is choosing between required, optional, and nullable described as "the most consequential decision in schema design" — and why does the wrong choice not throw an error but silently produce bad data?
82. When should `required` be used — only when you're certain the source will always provide the data?

83. When should `optional` be used — when the downstream system treats absence and null the same way?
84. When should `nullable` be used — when the downstream system needs an explicit signal that a field was checked and not found?
85. Why is a missing field described as "ambiguous" while a null value is described as "an honest answer"?

Enum Fields

86. What does an enum field eliminate that free-text categorization allows (invented labels, slight variations across runs)?
87. What is a "closed enum," and why does it work well only when the defined values cover every case that will ever appear?

The Closed Enum Problem

88. What happens when the source contains a value the closed enum didn't anticipate — does the model report an error, or force a bad match from the existing list?
89. Why is this failure described as "silent" — does the output pass schema validation even when the classification is wrong?
90. What happens over time to a closed enum as new categories appear in the data?

The "Other" Plus Detail Pattern

91. What does adding an "other" option paired with a free-text detail field solve that a plain closed enum cannot?
92. How does this pattern let known categories process automatically while routing unanticipated values to human review?
93. How can tracking the detail-field values in "other" cases over time reveal which new categories are emerging?

Representing Ambiguity

94. What is the difference between "other" (a value exists but doesn't fit the list) and "unclear" (the source doesn't contain enough information to classify at all)?
95. Why does forcing a category choice on a genuinely ambiguous source produce "confident but wrong classifications"?
96. What does tracking the volume of "unclear" results over time reveal about source data quality or enum design?

Format Normalization Rules

97. What is the difference between what a schema fixes (shape) and what a normalization rule fixes (values inside that shape)?
98. Why can't a schema catch inconsistent date formats, phone number formats, or name casing — even though all of them are "valid strings"?
99. For dates, contact info, currency, and text/category fields, what specific normalization rules does the guide recommend (ISO 8601, E.164, base-unit currency, ISO country codes)?

100. Why should a normalization rule say "if a date is missing, return null; do not infer a date" rather than leaving inference up to the model?

When Normalization Rules Are Not Enough

101. When source data is genuinely ambiguous, why can't a normalization rule resolve it — what should be used instead (nullable, or an "unclear" enum value)?
102. Why does a complex normalization rule benefit from a worked example showing before-and-after, rather than being stated as an instruction alone?
103. Why should normalization rules be placed in the prompt immediately after the schema definition rather than far away in a long prompt?

F. Validation, Retry, and Feedback Loops

104. What is a validation loop, and what happens to bad data if one doesn't exist?
105. Why does each step in the validation loop depend on the one before it — what happens if a step is skipped (e.g., validating semantics before validating structure)?

Resolvable vs. Unresolvable Errors

106. What is the fundamental difference between a resolvable error (data exists but in the wrong shape) and an unresolvable error (data is simply not in the source)?
107. Why does retrying a resolvable error without feedback just make the model "guess again"?
108. Why does retrying an unresolvable error push the model toward fabrication rather than fixing anything?

The Fabrication Risk

109. What is a "fabricated value," and why does it pass schema validation while still being wrong?
110. Why is a plausible-looking fabricated value described as "dangerous" — because it fails silently?
111. What is the only way to prevent fabrication from an unresolvable-error retry?

Retry with Error Feedback

112. What three things should be sent back to the model on a retry (original input, failed output, specific validation error)?
113. Why does "invoice_number is missing" give the model more to act on than "the output was invalid"?
114. Why must errors the model cannot fix be excluded from a feedback prompt that also contains fixable errors — what risk does mixing them create?

Validation Layers

115. What are the three validation layers (schema, semantic, business rule), and what specific class of error does each one catch?
116. Why does no single layer catch everything — what does schema validation miss that semantic validation catches, and what does semantic validation miss that business rule validation catches?

Schema Validation

117. What four things does schema validation catch (missing required fields, wrong types, values outside enum) — and what does it NOT catch (wrong values, inconsistent values, business rule violations)?

Semantic Validation

118. Why can't a schema catch line items that don't sum to a total, or a value placed in a wrong field with a matching type?
119. What is the difference between a syntactically valid record and a semantically consistent one?

Business Rule Validation

120. What kind of constraint is business rule validation for (a ship date after an order date, a discount not exceeding item price) that goes beyond simple arithmetic or field placement?

Semantic Validation Errors — Causes

121. What are the two main causes of a semantic error (extraction error where the model misplaced or miscopied a value, versus source inconsistency where the document itself is contradictory)?
122. Why is an extraction-error-caused semantic error often resolvable by retry, while a source-inconsistency-caused one usually isn't?

Adding Semantic Checks to Pydantic

123. What is the difference between a field validator (runs on one field) and a model validator (runs after all fields, with access to the whole record)?
124. Why does comparing a ship date to an order date require a model validator rather than a field validator?
125. Why does a Pydantic `ValidationError` message feed directly and usefully into a retry feedback prompt?

In-Schema Self-Checks

126. What is the purpose of a self-check field — capturing extracted data, or capturing the model's own assessment of that extraction?
127. What does the `calculated_total` field surface, and how is it compared against `stated_total`?
128. What does `conflict_detected` flag, and why can't the model resolve a source-level contradiction on its own?
129. What does `detected_pattern` reveal over time about which criteria produce false positives?
130. What does a low `confidence` value on a field indicate, and what should happen to that record?

Max Retry Limits

131. Why is a max retry limit "not optional" in production systems — what happens without one on an unresolvable error?
132. What is the recommended retry limit range for most extraction tasks (two to three attempts), and why do returns diminish beyond that?
133. What three things should happen to a record when it exceeds the retry limit (log it, route to human review, never let it silently enter downstream systems)?

134. Why is a record that "required three retries and still failed" considered unreliable even if its final output looks structurally valid?

Using the Retry Limit as a Diagnostic Signal

135. What does it signal when one specific field repeatedly hits the retry limit across many documents (a systemic schema or prompt problem for that field)?
136. What does it signal when one document type regularly exhausts retries (the format isn't covered by existing examples)?
137. What does it signal when the retry limit is hit on the very first attempt (the error is likely unresolvable from the start)?

G. Batch Processing Strategies

138. What is the fundamental trade the Message Batches API offers — time for money?

How the Message Batches API Works

139. What must each request in a batch submission carry (a `custom_id`, model, `max_tokens`, messages)?
140. What is the guaranteed maximum processing window, and why must a system be designed to handle the full window rather than the typical completion time?
141. Why must results be matched back to their source requests by `custom_id` rather than by position in the results file?

Batch vs. Synchronous Processing

142. What is the single deciding question for choosing batch versus synchronous — does anything stop working while waiting for the result?
143. Why is volume never the deciding factor on its own — what actually determines the choice (whether something is blocked)?

What Batch Processing Cannot Do

144. Why can't batch processing support multi-turn tool loops within a single request — what must agentic workflows use instead?
145. Why can't batch processing replace synchronous processing when an SLA requires results faster than 24 hours?

Failure Handling

146. Why is a `custom_id` necessary for matching results, given that batch results don't return in submission order?
147. Why must `custom_id` values be unique within a batch — what happens with duplicates?
148. Why does resubmitting only the failed items (not the whole batch) keep failure-handling cost proportional to the number of failures?
149. What is "chunking," and why must an oversized request be split rather than simply retried as-is?

150. What causes batch expiration during busy periods, and why does splitting one large batch into several smaller ones reduce this risk?

Submission Frequency and SLA Constraints

151. Why must a submission schedule account for the full 24-hour processing window rather than the typical under-1-hour completion time?
152. What is "buffer time," and why does a schedule with no buffer have "no recovery path" for failures?
153. Why does Anthropic's documentation recommend testing a single request shape with the synchronous Messages API before submitting a full batch?
154. Given a 100,000-request/256MB batch size limit, why must very large datasets be broken into multiple batches?
155. How long are batch results available for retrieval after batch creation, and what happens after that window closes?

H. Multi-Instance and Multi-Pass Review

156. What is the difference in purpose between multi-instance review (separate sessions for generation and review) and multi-pass review (splitting a large review into focused passes)?

Self-Review vs. Independent Review Instance

157. Why does a model that just generated an output still hold the reasoning it used, making it less likely to question its own decisions in the same session?
158. Why is "trusted to catch subtle issues it generated" listed as a common exam trap for same-session self-review?

Multi-Pass Review

159. What is the difference between a per-file local pass and a cross-file integration pass — what does each catch that the other misses?
160. Why does skipping the integration pass leave cross-file interaction bugs (like a renamed parameter with an unchanged caller) unfound?
161. What common failure does combining both passes fix (deep feedback on some files, shallow on others, self-contradiction on identical code)?

Confidence-Based Review Routing

162. Why does reporting a confidence level with each finding let a team focus limited reviewer time where it's most needed?
163. In the worked example (Phase 1 per-file, Phase 2 integration with a fresh instance, confidence-based routing), what does each phase specifically contribute?

Common Pitfalls

164. Why does running a single pass over many files "spread attention too thin"?
165. Why does requiring agreement across repeated full runs risk hiding bugs that are only caught some of the time, rather than confirming they're not real?

Cross-Cutting Self-Questioning Prompts for Domain 4

166. For any scenario where a rule must hold every single time without exception — is a clearer prompt being reached for when an architectural/deterministic mechanism is actually required?
167. For any scenario describing inconsistent output format or judgment calls — is this a case where few-shot examples would outperform additional zero-shot instructions?
168. For any scenario describing a field that's sometimes present and sometimes fabricated — is this a required-vs-nullable schema design mistake?
169. For any scenario describing a structurally valid but factually wrong output (wrong sum, wrong field, contradicting values) — is this a semantic error that schema validation alone cannot catch?
170. For any scenario describing a retry that seems to loop indefinitely or produce increasingly "plausible" wrong values — is this an unresolvable error being mistakenly retried?
171. For any scenario describing latency-tolerant, high-volume, non-blocking work — is the Message Batches API the fit, and does any mention of tool loops or blocking dependencies rule it out?
172. For any scenario describing subtle bugs that were "considered but dismissed" during generation — does this call for an independent review instance rather than same-session self-review?
173. For any scenario mixing multiple concern types (security, API design, business logic) in one review prompt where fixing one hurts another — is splitting into focused, separately-tuned prompts the fix?